Python Reference

Python Cheat Sheet

A complete quick-reference for Python syntax — 12 sections, 50+ copy-ready snippets. Whether you're studying for an interview, starting a new project, or just need a reminder, hover any block to copy it instantly.

12 topic sections **50+** code snippets **Free** to download

Download .py

Variables & Data Types

Variable assignment

```
name = "Alice"
age = 25
price = 9.99
is_active = True
```

Check the type

```
print(type(name))   # <class 'str'>
print(type(age))    # <class 'int'>
```

Type conversion

```
int("42")      # 42
float("3.14")  # 3.14
str(100)       # "100"
bool(0)        # False
```

Multiple assignment

```
x, y, z = 1, 2, 3
a = b = c = 0
```

Strings

Creating strings

```
single = 'Hello'
double = "World"
multi  = """Multiple
lines"""
```

f-strings (recommended)

```
name = "Alice"
age = 25
print(f"Hi, {name}! You are {age}.")
```

String methods

```
"hello".upper()          # "HELLO"
"HELLO".lower()          # "hello"
"  hi  ".strip()         # "hi"
"a,b,c".split(",")       # ["a", "b", "c"]
",".join(["a","b","c"])  # "a,b,c"
```

String slicing

```
s = "Python"
s[0]    # "P"   (first char)
s[-1]   # "n"   (last char)
s[0:3]  # "Pyt" (slice)
s[::-1] # "nohtyP" (reverse)
```

Check substring

```
"py" in "Python"          # False
"Py" in "Python"          # True
"Python".startswith("Py") # True
"Python".endswith("on")   # True
```

Numbers

Arithmetic

```
10 + 3   # 13
10 - 3   # 7
10 * 3   # 30
10 / 3   # 3.333 (float division)
10 // 3  # 3     (floor division)
10 % 3   # 1     (remainder)
2 ** 8   # 256   (power)
```

Math module

```
import math
math.sqrt(16)   # 4.0
math.pi         # 3.14159...
math.ceil(4.1)  # 5
math.floor(4.9) # 4
math.abs(-7)    # 7 (use abs(-7))
```

Random numbers

```
import random
random.randint(1, 10)     # 1-10 inclusive
random.random()           # 0.0-1.0
random.choice(["a","b"])  # random item
```

Lists

Create & access

```
fruits = ["apple", "banana", "cherry"]
fruits[0]    # "apple"
fruits[-1]   # "cherry"
len(fruits)  # 3
```

Modify

```
fruits.append("date")          # add to end
fruits.insert(1, "avocado")    # insert at index
fruits.remove("banana")        # remove by value
fruits.pop()                   # remove & return last
fruits.pop(0)                  # remove & return index 0
```

Slice & sort

```
nums = [3, 1, 4, 1, 5]
nums[1:3]     # [1, 4] — slice
nums.sort()   # sort in-place: [1, 1, 3, 4, 5]
sorted(nums)  # new sorted list
```

List comprehension

```
squares = [x**2 for x in range(1, 6)]
# [1, 4, 9, 16, 25]

evens = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]
```

Useful operations

```
3 in [1, 2, 3]       # True
sum([1, 2, 3])        # 6
min([1, 2, 3])        # 1
max([1, 2, 3])        # 3
list(range(5))        # [0, 1, 2, 3, 4]
```

Dictionaries

Create & access

```
person = {"name": "Alice", "age": 25}
person["name"]          # "Alice"
person.get("email")     # None (no error)
person.get("email","?") # "?" (default)
```

Modify

```
person["email"] = "a@b.com"  # add/update
del person["age"]             # delete key
person.pop("email", None)     # remove safely
```

Iterate

```
for key in person:
    print(key, person[key])

for key, value in person.items():
    print(f"{key}: {value}")
```

```
list(person.keys())    # ["name", "email"]
list(person.values())  # ["Alice", "a@b.com"]
```

Dict comprehension

```
squares = {x: x**2 for x in range(1, 6)}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Control Flow

if / elif / else

```
score = 75

if score ⩾ 90:
    print("A")
elif score ⩾ 70:
    print("B")   # this runs
else:
    print("C")
```

Comparison operators

```
== ≠ < > ⩽ ⩾
is    # identity (is None)
is not
in    # membership
not in
```

Logical operators

```
True and False  # False
True or False   # True
not True        # False
```

Ternary expression

```
status = "pass" if score ⩾ 70 else "fail"
```

Loops

for loop

```
for i in range(5):        # 0, 1, 2, 3, 4
    print(i)

for item in ["a","b","c"]:
    print(item)
```

while loop

```
count = 0
while count < 3:
    print(count)  # 0, 1, 2
    count += 1
```

Loop control

```
for i in range(10):
    if i == 3: continue  # skip 3
    if i == 7: break     # stop at 7
    print(i)
```

enumerate & zip

```
# Index + value
for i, val in enumerate(["a","b","c"]):
    print(i, val)   # 0 a, 1 b, 2 c

# Pair two lists
for a, b in zip([1,2,3], ["x","y","z"]):
    print(a, b)     # 1 x, 2 y, 3 z
```

Functions

Define & call

```
def greet(name):
    return f"Hello, {name}!"

result = greet("Alice")
print(result)  # Hello, Alice!
```

Default parameters

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"
```

```
greet("Bob")            # Hello, Bob!
greet("Bob", "Hi")      # Hi, Bob!
```

Keyword arguments

```
def power(base, exp=2):
    return base ** exp

power(3)           # 9
power(3, exp=3)    # 27
power(base=2, exp=8)  # 256
```

*args and **kwargs

```
def add(*nums):
    return sum(nums)

add(1, 2, 3, 4)   # 10

def info(**kwargs):
    for k, v in kwargs.items():
        print(f"{k}: {v}")
```

Lambda functions

```
double = lambda x: x * 2
double(5)     # 10

sorted_list = sorted([3,1,2], key=lambda x: -x)
# [3, 2, 1]
```

Error Handling

try / except

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
except (TypeError, ValueError) as e:
    print(f"Error: {e}")
else:
    print("No error!")
finally:
    print("Always runs")
```

Raise an error

```
def divide(a, b):
    if b == 0:
        raise ValueError("Divisor cannot be zero")
    return a / b
```

Common exceptions

```
TypeError       # wrong type
ValueError      # correct type, wrong value
IndexError      # list index out of range
KeyError        # dict key missing
NameError       # name not defined
FileNotFoundError  # file missing
ZeroDivisionError  # divide by zero
```

File I/O

Read a file

```
with open("data.txt", "r") as f:
    content = f.read()     # entire file as string
    # or:
    lines = f.readlines()  # list of lines
```

Write to a file

```
with open("output.txt", "w") as f:
    f.write("Hello, World!
")

# Append mode
with open("log.txt", "a") as f:
    f.write("New entry
")
```

Classes & OOP

Define a class

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed
```

```
    def bark(self):
        return f"{self.name} says: Woof!"

fido = Dog("Fido", "Labrador")
print(fido.bark())
```

Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."

class Cat(Animal):
    def speak(self):
        return f"{self.name} says: Meow!"

kitty = Cat("Kitty")
kitty.speak()   # Kitty says: Meow!
```

Useful Built-ins

Frequently used

```
len([1,2,3])     # 3 — length
range(5)         # 0,1,2,3,4
type(42)         # <class 'int'>
print("hello")   # output to console
input("Name: ")  # get user input (returns str)
abs(-5)          # 5
round(3.7)       # 4
sorted([3,1,2])  # [1, 2, 3]
reversed([1,2,3]) # iterator
zip([1,2],[3,4]) # [(1,3),(2,4)]
enumerate(["a"]) # [(0,"a")]
```

String formatting

```
# f-string (Python 3.6+) — preferred
f"Result: {10 * 3}"

# .format()
"Hello, {}!".format("Alice")
```

```
# % formatting (old style)
"Hello, %s!" % "Alice"
```

Getting an error message?

Our Python errors guide explains the most common exceptions with clear examples and fixes.

Python Errors Guide

Ready to practise?

Run Python code directly in your browser — no setup needed. Our free course teaches everything in this cheat sheet step by step.

Start Free Python Course | Python Playground

EXPLORE MORE

› Python Glossary › Python Built-in Functions › Python Errors Hub
› Beginner Roadmap › Python Projects › Python Beginner Guide
› Learn Python Course › Data Analysis Course › Flask Web Dev Course › SQL Course
› Advanced Python › Python Articles › Python Playground › Dev Tools
› Pro Membership › For Schools › About OpenPython